ISOM5240 — Deep Learning Business Applications with Python

HSBC Banking Query Intent Router

Group 02 | SHEN, Jia (21158135) | TSAI Chungkai (21182344)

1. Project Overview & Metadata

Executive Summary

This project implements and evaluates two text-based intent pipelines on the benchmark *BANKING77* dataset: a sequence classifier fine-tuned using distilbert-base-uncased and an embedding-retrieval baseline using *all-MiniLM-L6-v2*.

Additionally, an integrated voice-input mode utilizes *openai/whisper-tiny* to transcribe browser-recorded spoken enquiries before routing them to the *DistilBERT* classifier. Evaluated across all 3,080 official test examples, the retrieval baseline achieved **87.18%** accuracy and the fine-tuned classifier achieved **83.18%**. The deployed Streamlit prototype successfully operationalizes both text and voice interaction modes.

Project Objective

This project aims to automate online-banking query routing using deep learning techniques to triage customer support messages across 77 service intents, facilitating **consistent query handling** and **data-driven decision-making** at HSBC.

Project Parameters & Links:

- **Target Organization:** HSBC — https://www.hsbc.com
- **GitHub Repository:** https://github.com/elva329/Group02_21158135
- **Public Streamlit App URL:** https://group02.streamlit.app/
- **Public Fine-Tuned Model URL:** https://huggingface.co/elva329/banking77-distilbert

2. Business Problem and Strategy

The Business Problem

Online banking support channels process a high volume of diverse customer enquiries daily spanning dozens of specialized domains (such as card management, transaction disputes, and account security). Online banking enquiries can concern many different services, and identifying the service involved is a necessary first step in routing. Manually sorting and routing these messages creates significant operational overhead and potential delays.

To address this challenge, the prototype assigns an incoming English message to one of *BANKING77*'s intent categories, such as a card, transfer, or cash-withdrawal issue.

Methodological Strategy

To solve this routing problem using deep learning, the implementation follows these structured phases:

1. **Data Preparation:** Utilize *BANKING77*'s supplied training and test partitions, reserving a validation subset from the training data via stratified splitting using seed 42.
2. **Supervised Fine-Tuning (Pipeline A)**: Fine-tune a pretrained distilbert-base-uncased checkpoint for the 77-class sequence classification task.
3. **Embedding Retrieval (Pipeline B)**: Build a zero-shot baseline using a pretrained *all-MiniLM-L6-v2* encoder and similarity-ranked reference training messages.
4. **Empirical Evaluation**: Measure both approaches on the untouched test partition (3,080 examples), comparing accuracy, per-message inference runtime, and setup indexing overhead.
5. **Application Deployment**: Expose the models through an interactive Streamlit interface supporting text classification, similar-example retrieval, and *Whisper Tiny* voice-to-text transcription.

3. Dataset and Pre-processing

The model training and evaluation utilize the benchmark dataset structured as follows:

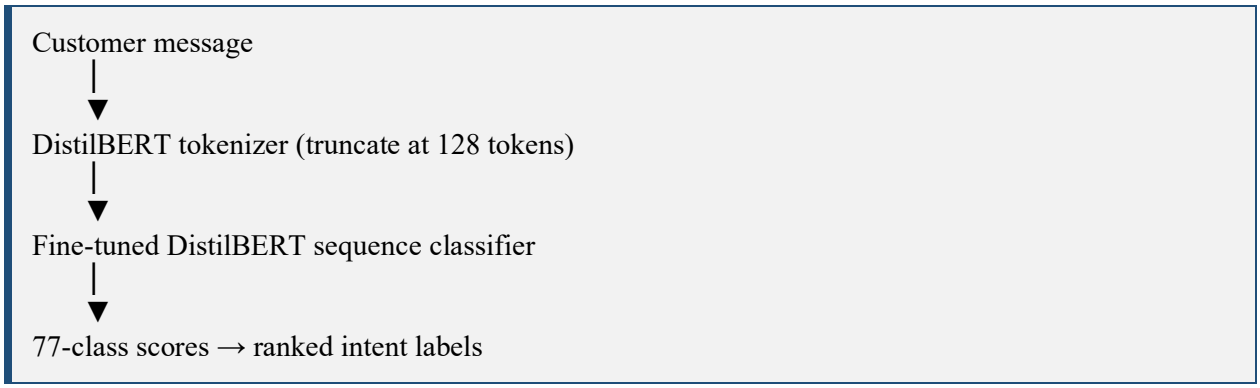
Dataset Attribute	Specification & Description
Dataset Source	PolyAI/BANKING77 dataset card (https://huggingface.co/datasets/PolyAI/banking77)
Total Volume	13,083 English banking-support messages
Label Taxonomy	77 distinct intent categories
Relevant Features	text (customer message) and label (integer class); intent names are mapped for readable output.
Training Subset	9,002 messages used for fine-tuning.
Validation Subset	1,001 messages, reserved from the official training partition using a stratified split and seed 42.
Testing Subset	3,080 messages from the official test partition, retained for final evaluation.
Retrieval Reference Set	Up to 25 training messages for each intent, selected reproducibly with seed 42 (up to 1,925 reference messages).
Licensing & Attribution	CC BY 4.0. Attribution is preserved and dataset card terms checked before redistribution.

Text inputs are tokenized and truncated at 128 tokens for the classifier. Padding is applied dynamically within training batches. The retrieval pipeline mean-pools token embeddings using the attention mask and L2-normalizes each vector. Test messages are excluded from the retrieval reference index.

4. Models and System Design

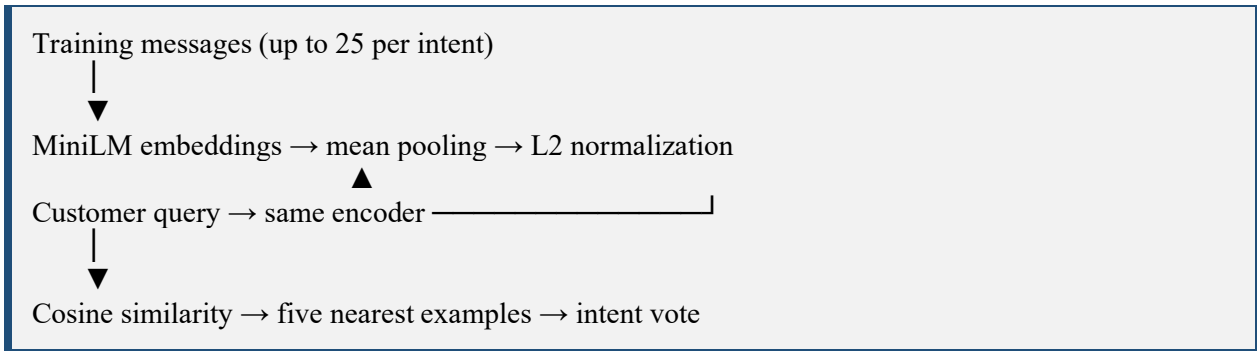
Pipeline A — Fine-Tuned Intent Classifier

The training program initializes *distilbert/distilbert-base-uncased* (https://huggingface.co/distilbert/distilbert-base-uncased) as a sequence-classification model with 77 outputs. It trains for three epochs using learning rate 2e-5, weight decay 0.01, batch sizes of 16 for training and 32 for evaluation, and random seed 42. The checkpoint with the highest validation accuracy is retained.



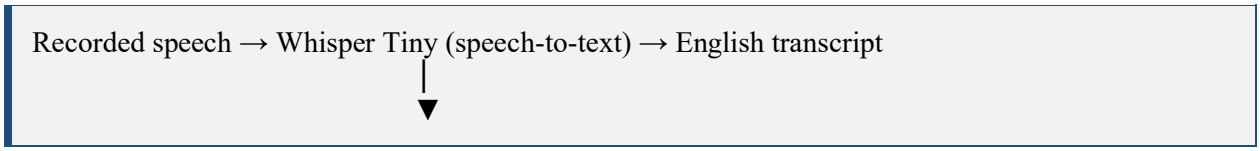
Pipeline B — Pretrained Embedding Retrieval

This pipeline uses *sentence-transformers/all-MiniLM-L6-v2* (https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2) as a pretrained feature extractor; it is not fine-tuned in this project. The query vector is compared by cosine similarity with the balanced reference vectors. The five closest examples vote on the intent; votes are ranked first, with average similarity breaking ties.



Voice Interaction

Voice-input mode serves as an audio-to-text extension of the *DistilBERT* intent-classification pipeline. The user records speech in the browser; the pretrained Hugging Face *whisper-tiny* (https://huggingface.co/openai/whisper-tiny) model transcribes it into English text, and the transcript is passed to the fine-tuned DistilBERT classifier. Whisper performs speech recognition, while *DistilBERT* performs intent classification.



Fine-tuned DistilBERT → Predicted intent

The reported model comparison evaluates the two text-based intent pipelines; voice-mode accuracy was not measured separately.

5. Experiments and Results

Both intent pipelines were evaluated on the complete official test partition (3,080 messages). Model selection evaluates accuracy and inference runtime. The empirical results below are extracted from *Python_notebooks/results/banking77_comparison.csv*.

Pipeline	Accuracy	Test-set Inference Runtime	Inference Runtime / Msg	Setup / Index Runtime
Fine-tuned DistilBERT	83.18%	3.278 s	1.064 ms	0.189 s
Pretrained MiniLM Retrieval	87.18%	1.708 s	0.554 ms	2.102 s

Retrieval outperformed the fine-tuned classifier by **3.99** percentage points in accuracy and demonstrated a lower per-message inference runtime, at the expense of a one-time reference indexing setup (2.102 s).

6. Deployment and Application Verification

The Streamlit prototype (*Group02_program/GitHub_App_Files/app.py*) exposes three primary operational modes:

- **Classify my message**: Evaluates input via fine-tuned *DistilBERT* to predict primary and alternative intents.
- **Find similar examples**: Executes *MiniLM* embedding retrieval to surface nearest reference queries and similarity scores.
- **Speak my message**: Transcribes browser-recorded audio via *Whisper Tiny* before intent classification.

Application Interface Screenshots

Visual verification of the deployed Streamlit Cloud application in operation:

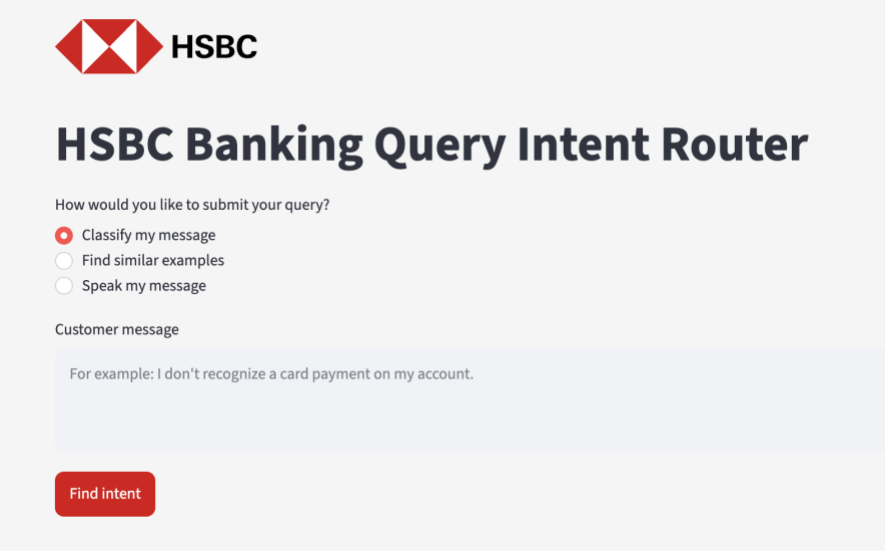


Figure 1: Streamlit Cloud App Default Interface

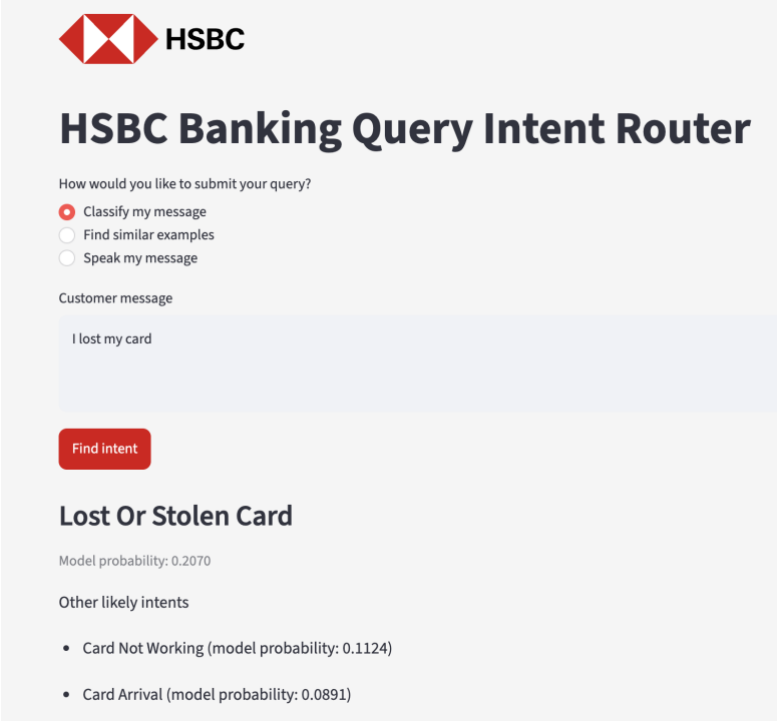


Figure 2: Text Classification Mode showing customer query input and predicted intent scores



Figure 3: Similar-Example Retrieval & Voice Input Transcription Mode

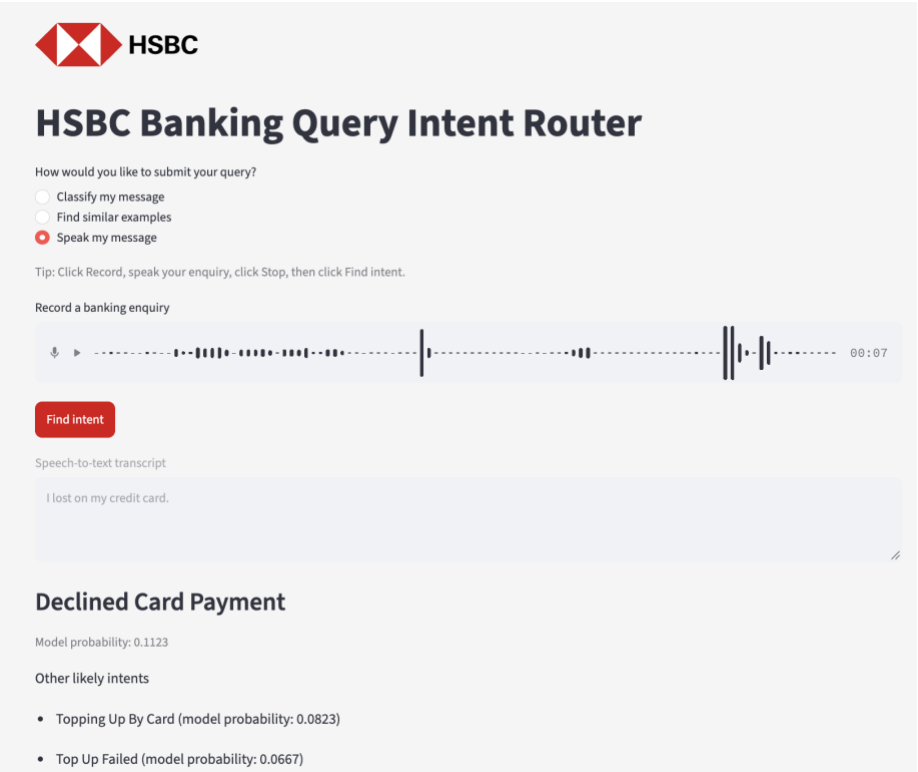


Figure 4: Multi-stage voice interaction workflow

7. Conclusion, Limitations, and Implementation Roadmap

(a) Evaluation of Business Problem Resolution

The technical benchmark objectives were successfully achieved. The system classifies multi-intent banking queries into 77 service categories, with embedding retrieval achieving 87.18% test accuracy and fine-tuned DistilBERT achieving 83.18%. However, the business objective is only partially satisfied: while the prototype validates automated intent triage feasibility, HSBC-specific operational performance, domain-specific terminology adaptation, and live customer routing efficacy remain unverified.

(b) Production Implementation Roadmap

To successfully transition the prototype into an enterprise production environment for HSBC support workflows, the following implementation framework is required:

- **Public Cloud Hosting & Model Publication**: Finalize deployment of the Streamlit application on Community Cloud and publish fine-tuned checkpoint weights on Hugging Face.
- **Domain-Specific Fine-Tuning & Governance**: Retrain and validate models using governance-compliant, representative HSBC customer interaction datasets to handle proprietary banking terminology and regional policy variations.
- **Confidence Thresholding & Human-in-the-Loop**: Implement probabilistic confidence cutoffs where low-confidence predictions automatically route to human customer service agents.
- **Enterprise API Integration**: Securely integrate the inference engine into HSBC's customer relationship management (CRM) and ticket-routing infrastructure via encrypted RESTful APIs.
- **Security & Compliance Audits**: Conduct comprehensive data privacy, threat modeling, and regulatory compliance assessments prior to operational deployment.

8. Sources

[1] Hugging Face. (n.d.). *Distilbert-base-uncased*. Retrieved from https://huggingface.co/distilbert/distilbert-base-uncased

[2] Hugging Face. (n.d.). *Openai/whisper-tiny*. Retrieved from https://huggingface.co/openai/whisper-tiny

[3] Hugging Face. (n.d.). *Sentence-transformers/all-MiniLM-L6-v2*. Retrieved from https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[4] HSBC. (n.d.). *HSBC corporate website*. Retrieved from https://www.hsbc.com

[5] PolyAI. (n.d.). *Banking77 dataset*. Hugging Face. https://huggingface.co/datasets/PolyAI/banking77